

Hands-On Lab 4 : Understanding Git Merge Conflicts

Learning Objective

- Understand what a merge conflict is.
 - Learn why conflicts happen.
 - Practice resolving merge conflicts manually.
 - Complete a successful conflict resolution.
-

Learning Outcome

After completing this lab, you will be able to:

- Identify conflict markers.
 - Understand why Git cannot auto-merge.
 - Manually resolve conflicts.
 - Commit resolved changes properly.
-

Concept Explanation

What is a Merge Conflict?

A merge conflict happens when: Two branches modify the same line of the same file.

Git cannot decide which change is correct. So it asks YOU to resolve it.

Why Do Conflicts Happen?

✓ Multiple developers editing same file ✓ Same lines modified differently ✓ Long-running branches

Lab Steps

Step 1 : Create Repository

```
mkdir conflicts  
cd conflicts  
git init
```

Create file:

```
vi loginfile  
git add loginfile  
git commit -m "master user commit"
```

Step 2 : Create Branches

```
git branch dev1  
git branch dev2
```

Step 3 : Modify in dev 1

```
git switch dev1  
vi loginfile  
git add loginfile  
git commit -m "dev1 commit"
```

Step 4 : Modify in dev 2

```
git switch dev2  
vi loginfile  
git add loginfile  
git commit -m "dev2 commit"
```

Step 5 : Merge dev 1 into master

```
git switch master  
git merge dev1
```

Step 6 : Merge dev 2 (Conflict Happens)

```
git merge dev2
```

You will see conflict markers inside loginfile:

```
<<<<<<< HEAD  
Code from master  
=====  
Code from dev2  
>>>>>>> dev2
```

How to Resolve Conflict

- 1 . Open the file.
- 2 . Remove conflict markers.
- 3 . Keep correct code.
- 4 . Save the file.

Then:

```
git add loginfile  
git commit
```

Important Understanding

Marker	Meaning
<<<<<<<	Current branch
=====	Separator
>>>>>>>	Incoming branch

